An Industrial Oriented Mini Project / Summer Internship Report on

IDENTIFICATION OF DIABETIC RETINOPATHY USING SUPERVISED LEARNING MODEL

Submitted in Partial fulfillment of requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DATA SCIENCE

By

B. SAKETH REDDY	**20BD1A6707**
B. ROHIT	**20BD1A6706**
GANUMALA LAVITRA LALIT	**20BD1A6721**
ATHARVA RAJE	**20BD1A6704**

Under the guidance of

Dr. Vishal Reddy
Assistant Professor, Department of CSD

DEPARTMENT OF DATA SCIENCE

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTION)
Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH.
Narayanaguda, Hyderabad, Telangana-29

2023-24

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTION)
Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH.
Narayanaguda, Hyderabad, Telangana-29

DEPARTMENT OF DATA SCIENCE

CERTIFICATE

This is to certify that this is a bonafide record of the project report titled **"IDENTIFICATION OF DIABETIC RETINOPATHY USING SUPERVISED LEARNING MODEL"** which is being presented as the Industrial Oriented Mini Project / Summer Internship report by

1. **B. SAKETH REDDY**	**20BD1A6707**
2. **B. ROHIT**	**20BD1A6706**
3. **GANUMALA LAVITRA LALIT**	**20BD1A6721**
4. **ATHARVA RAJE**	**20BD1A6704**

In partial fulfillment for the award of the degree of Bachelor of Technology in Data Science affiliated to the Jawaharlal Nehru Technological University Hyderabad, Hyderabad

Internal Guide
(Dr. Vishal Reddy)

Head of Department
(Mr. K. Anil Kumar)

Submitted for Viva Voce Examination held on _____

External Examiner

Vision of KMIT

- To be the fountainhead in producing highly skilled, globally competent engineers.
- Producing quality graduates trained in the latest software technologies and related tools and striving to make India a world leader in software products and services.

Mission of KMIT

- To provide a learning environment that inculcates problem solving skills, professional, ethical responsibilities, lifelong learning through multi modal platforms and prepares students to become successful professionals.
- To establish an industry institute Interaction to make students ready for the industry.
- To provide exposure to students on the latest hardware and software tools.
- To promote research-based projects/activities in the emerging areas of technology convergence.
- To encourage and enable students to not merely seek jobs from the industry but also to create new enterprises.
- To induce a spirit of nationalism which will enable the student to develop, understand India's challenges and to encourage them to develop effective solutions.
- To support the faculty to accelerate their learning curve to deliver excellent service to students.

Vision of CSD Department

To produce globally competent graduates to meet the modern challenges through contemporary knowledge and moral values committed to build a vibrant nation.

Mission of CSD Department

- To create an academic environment, which promotes the intellectual and professional development of students and faculty.
- To impart skills beyond university prescribed to transform students into a well-rounded CSD professional.
- To nurture the students to be dynamic, industry ready and to have multidisciplinary skills including e-learning, blended learning and remote testing as an individual and as a team.
- To continuously engage in research and projects development, strategic use of emerging technologies to attain self-sustainability.

PROGRAM OUTCOMES (POs)

1. **Engineering Knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. **Problem Analysis:** Identify formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences

3. **Design/Development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. **Conduct Investigations of Complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretationof data, and synthesis of the information to provide valid conclusions.

5. **Modern Tool Usage:** Create select, and, apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

6. **The Engineer and Society:** Apply reasoning informed by contextual knowledge to societal, health, safety. Legal und cultural issues and the consequent responsibilities relevant to professional engineering practice.

7. **Environment and Sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts and demonstrate the knowledge of, and need for sustainable development.

8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. **Individual and Team Work:** Function effectively as an individual, and as a member or leader in diverse teams and in multidisciplinary settings.

10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. **Project Management and Finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. **Life-Long Learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: An ability to analyze the common business functions to design and develop appropriate Information Technology solutions for social upliftments.

PSO2: Shall have expertise on the evolving technologies like Python, Machine Learning, Deep learning, IOT, Data Science, Full stack development, Social Networks, Cyber Security, Mobile Apps, CRM, ERP, Big Data, etc.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO1: Graduates will have successful careers in computer related engineering fields or will be able to successfully pursue advanced higher education degrees.

PEO2: Graduates will try and provide solutions to challenging problems in their profession by applying computer engineering principles.

PEO3: Graduates will engage in life-long learning and professional development by rapidly adapting to the changing work environment.

PEO4: Graduates will communicate effectively, work collaboratively and exhibit high levels of professionalism and ethical responsibility.

PROJECT OUTCOMES

P1: Study Convolutional Neural Network techniques to build a classification model.

P2: To examine the features of a patient suffering from diabetic retinopathy

P3: Build a classification model to identify if a diabetic patient is infected with this disease.

P4: Identifying the best available method based on various models available in the market and compare them based on the performance metrics by team work and research.

MAPPING PROJECT OUTCOMES WITH PROGRAM OUTCOMES

PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
P1	H	H	M	H	M	L	M	L	H	M	M	M
P2	H	H	M	M	M	L	M	L	H	M	M	M
P3	H	H	M	M	M	L	M	L	H	M	M	M
P4	H	H	M	M	M	L	M	L	H	M	M	M

L – LOW M –MEDIUM H– HIGH

PROJECT OUTCOMES MAPPING PROGRAM SPECIFIC OUTCOMES

PSO	PSO1	PSO2
P1	H	H
P2	H	M
P3	M	M
P4	M	M

PROJECT OUTCOMES MAPPING PROGRAM EDUCATIONAL OBJECTIVES

PEO	PEO1	PEO2	PEO3	PEO4
P1	H	H	M	H
P2	M	H	M	H
P3	M	H	M	H
P4	M	H	M	H

DECLARATION

We hereby declare that the results embodied in the dissertation entitled **"IDENTIFICATION OF DIABETIC RETINOPATHY USING SUPERVISED LEARNING MODEL"** has been carried out by us together during the academic year 2023-24 as a partial fulfillment of the award of the B.Tech degree in Data Sciencefrom JNTUH. We have not submitted this report to any other university or organization for the award of any other degree.

Student Name	Rollno.
B. SAKETH REDDY	**20BD1A6707**
B. ROHIT	**20BD1A6706**
GANUMALA LAVITRA LALIT	**20BD1A6721**
ATHARVA RAJE	**20BD1A6704**

ACKNOWLEDGEMENT

We take this opportunity to thank all the people who have rendered their full support to our project work. We render our thanks to **Dr. B L Malleswari**, Principal who encouraged us to do the Project.

We are grateful to **Mr. Neil Gogte**, Founder & Director, **Mr. S. Nitin,** Director for facilitating all the amenities required for carrying out this project.

We express our sincere gratitude to **Ms. Deepa Ganu**, Director Academic for providing an excellent environment in the college.

We are also thankful to **Mr. K Anil Kumar**, Head of the Department for providing us with time to make this project a success within the given schedule.

We are also thankful to our guide **Dr. Vishal Reddy**, for his valuable guidance and encouragement given to us throughout the project work.

We would like to thank the entire Data Science Department faculty, who helped us directly andindirectly in the completion of the project.

We sincerely thank our friends and family for their constant motivation during the project work.

Student Name	Roll no.
B. SAKETH REDDY	20BD1A6707
B. ROHIT	20BD1A6706
GANUMALA LAVITRA LALIT	20BD1A6721
ATHARVA RAJE	20BD1A6704

ABSTRACT

Diabetic retinopathy is a prevalent and potentially sight-threatening complication of diabetes, affecting millions of individuals worldwide. Early diagnosis and timely intervention are crucial in preventing vision loss, making automated detection methods essential. This project presents an innovative approach for the automated detection of diabetic retinopathy using supervised learning and deep neural networks. Our study leverages a diverse dataset of retinal images, encompassing various stages of diabetic retinopathy. We preprocess the images, applying techniques such as image enhancement and segmentation to extract relevant features. We employ a deep neural network architecture, specifically a convolutional neural network (CNN), which has proven to be highly effective in image classification tasks. The supervised learning process involves training the CNN on the labeled dataset, with the network learning to differentiate between healthy retinas and those affected by diabetic retinopathy. We utilize a variety of evaluation metrics, including sensitivity, specificity, and AUC-ROC, to assess the model's performance. The developed model exhibits promising results in terms of accuracy, sensitivity, and specificity. This project contributes to the field by demonstrating the potential of deep learning techniques in diabetic retinopathy detection. The automated system we propose has the potential to assist healthcare professionals in early diagnosis and monitoring of diabetic retinopathy, thereby improving patient outcomes. It also reduces the subjectivity of human assessments, making the diagnostic process more consistent and scalable. In conclusion, our approach highlights the effectiveness of supervised learning and deep neural networks in the detection of diabetic retinopathy from retinal images. With further refinement and validation, this system holds the promise of becoming a valuable tool in the fight against this prevalent and sight-threatening disease.

LIST OF ABBREVIATIONS

DR	Diabetic Retinopathy
CNN	Convolutional Neural Network
AUC-ROC	Area Under the Curve – Receiver Operating Characteristic
DL	Deep Learning
ML	Machine Learning

LIST OF DIAGRAMS

LIST OF SCREENSHOTS

CONTENTS

CHAPTER - 1

1. INTRODUCTION

1.1 Purpose of the project

Diabetic Retinopathy emerges as a consequence of prolonged diabetes, presenting as an ocular complication intricately linked with this metabolic disorder. Notably, around 80% of individuals enduring diabetes for more than a decade exhibit varying stages of this ailment. Furthermore, the duration of diabetes directly influences the probability of diabetic retinopathy (DR) affecting an individual's visual faculties. Recent research underscores the significant role of DR in contributing to roughly 5% of total cases of blindness. The World Health Organization (WHO) estimates that approximately 347 million people worldwide have received a diabetes diagnosis, and a significant percentage, roughly 40-45%, showcases some manifestation of diabetic retinopathy. A visual examination of the image below unmistakably highlights the stark disparities between images captured from a healthy eye and those originating from an eye afflicted by DR. Several contributing factors come into play concerning the onset of this condition, encompassing the duration of diabetes, suboptimal management of blood sugar levels, and occurrences like pregnancy. Importantly, scientific investigations have illuminated the fact that early detection of DR can markedly decelerate, if not entirely avert, the progression toward visual impairment. Despite the substantial prevalence of diabetes cases, the predominant method for diagnostic evaluation remains a labor-intensive, manual process carried out by trained healthcare professionals. Regrettably, this process frequently results in delayed findings and treatment due to communication challenges and extended waiting times. Consequently, the primary objective of this project revolves around the implementation of an automated, highly efficient, and sophisticated approach, harnessing the power of image processing and pattern recognition techniques. The overarching goal is to facilitate the timely and precise identification of diabetic retinopathy, ultimately mitigating potential harm to the retina.

1.2 Problems with Existing System:

There are several challenges and problems associated with existing systems for detecting diabetic retinopathy using deep learning:

Deep learning models require large and diverse datasets for training. However, in the case of diabetic retinopathy, acquiring a sufficiently large dataset with well-labeled images is a challenge. Additionally, there is often an imbalance between the number of normal and diseased cases, which can bias the model's performance. Deep neural networks, especially deep convolutional neural networks (CNNs), are often considered "black-box" models, making it challenging to understand how they arrive at their decisions. In medical applications, interpretability is crucial for gaining trust and acceptance from healthcare professionals. Models trained on a specific population may not generalize well to other ethnic or age groups. Diabetic retinopathy can manifest differently in various populations, and models should be robust to such differences. The quality of retinal images can vary widely, leading to issues such as image artifacts, low contrast, or blurriness. Noise in the data can adversely affect model performance.

Deep learning models are prone to overfitting when the dataset is small or noisy. Overfit models do not generalize well to unseen data, potentially leading to incorrect diagnoses. Training deep neural networks often requires significant computational resources, which can be a challenge for healthcare institutions with limited access to high-performance computing. The use of deep learning in medical diagnostics is subject to regulatory and ethical considerations. Ensuring patient privacy and compliance with healthcare regulations is crucial.

Deploying deep learning models in clinical settings can be challenging. They must seamlessly integrate with the existing clinical workflow, and healthcare providers need to trust the results. Medical knowledge evolves over time, and models should be capable of adapting to new information and changing disease patterns. The cost of implementing deep learning-based systems, including the required equipment and software, can be a barrier to adoption, especially in resource-constrained healthcare environments. Addressing these problems requires ongoing research and development in the field of diabetic retinopathy detection using deep learning. Solutions include improving data quality, developing interpretable models, enhancing generalization, addressing ethical and regulatory concerns, and making the technology more accessible to a wider range of healthcare provider.

1.3 Proposed System:

Diabetic retinopathy causes blindness. Diabetic retinopathy is one of the eye diseases which is caused due to retinal blood vessel extraction. Diabetic retinopathy affects blood vessels in the light-sensitive tissue called the retina that lines the back of the eye. It is the most common cause of vision loss among people with diabetes and the leading cause of vision impairment and blindness among working-age adults. Here we proposed a system where we extract retinal blood vessels for detecting eye diseases. Manually extracting the retinal blood vessels is a long task. There are many automated methods available which makes work easier. Here proposed an algorithm to extract blood vessels from images. We used filtering methods to remove noise and environmental interference from the image. Local entropy thresholding and alternative sequential filter methodology has been adopted in this system. We had implemented this system in matlab. User will input a retina image into the system. System will apply filtering techniques. Image pre-processing steps are applied to get accurate results. System will remove all unwanted objects from the image. System will apply a DenseNet-121 model to extract retinal blood vessels. Finally, the system will detect diabetic retinopathy.

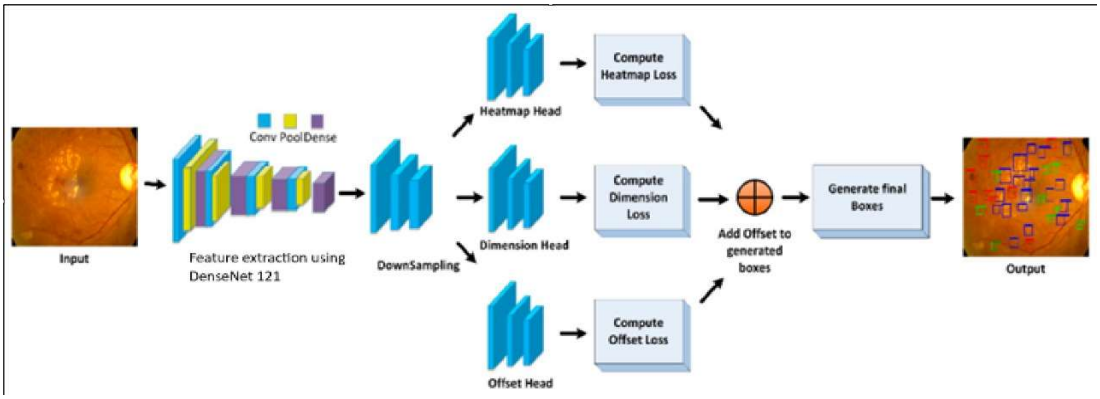


Figure 1.1: Architecture diagram

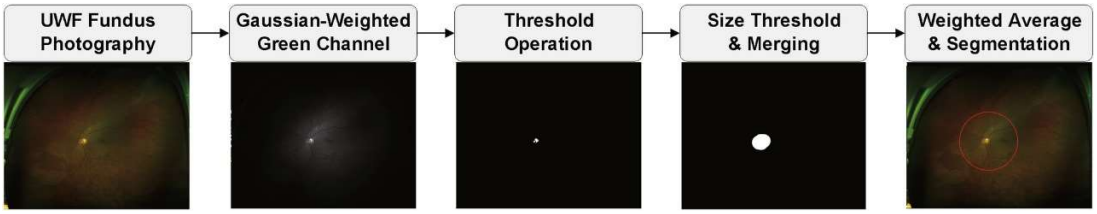


Figure 1.2: Input stages

Our DR detection system utilizes the DenseNet-121 model for the classification task since our dataset is relatively small and it is binary class data. The DenseNet-121 model utilized in our system is pre-trained on ImageNet23, and finetuned on the in-house dataset. Figure 5 illustrates the DenseNet-121 model. The DenseNet architecture provides advantages in an easier optimization and accuracy gain for deep networks21. To handle data imbalance between classes, we utilize weighted loss based on the number of training samples in the minority class (N). Weight for each class is obtained by dividing N by the number of training samples in each class. For optimization, the stochastic gradient descent with 0.001 learning rate and 0.9 momenta is utilized while the learning rate is set to decay by a factor of 0.1 for every 7 epochs. The number of epochs is set at 25.

Building an efficient deep neural network model using DenseNet-121 for the detection of diabetic retinopathy involves several steps. DenseNet-121 is a popular convolutional neural network architecture known for its effectiveness in image classification tasks. Here's a guide to building such a model:

1. Data Collection and Preparation:

Gather a dataset of retinal images, including both diabetic retinopathy cases and healthy retinas. Ensure the dataset is well-labeled and diverse.

Preprocess the data, including resizing images to a consistent size, normalizing pixel values, and handling class imbalances if necessary.

2. Data Augmentation:

Augment the dataset to increase its size and diversity. Apply data augmentation techniques such as rotation, flipping, zoom, and brightness adjustments.

3. Model Selection:

Choose the DenseNet-121 architecture as your base model. You can use a pre-trained DenseNet-121 model from libraries like PyTorch or TensorFlow.

4. Model Fine-Tuning:

Remove the original classification head of the pre-trained DenseNet-121 model and replace it with a new output layer suitable for your diabetic retinopathy detection task. This new output layer should have the appropriate number of classes (e.g., 2 for binary classification).

5. Regularization Techniques:

Implement regularization techniques, such as dropout and L2 regularization, to prevent overfitting. Adjust the dropout rate and weight decay based on your dataset and model performance.

6. Loss Function:

Use an appropriate loss function for classification tasks, like categorical cross-entropy. You can also consider using class-weighted loss to address class imbalances if necessary.

7. Optimization Algorithm:

Choose an optimization algorithm, such as Adam or RMSprop, for efficient model training. Experiment with different learning rates to find the best one for your dataset.

8. Training:

Train your DenseNet-121 model on the preprocessed dataset. Monitor the training process by observing metrics like loss and accuracy.

9. Model Evaluation:

Split your dataset into training, validation, and testing sets. Use the validation set to fine-tune hyperparameters and monitor the model's performance.

10. Metrics:

Select appropriate evaluation metrics, including accuracy, sensitivity, specificity, and area under the ROC curve (AUC-ROC), to assess the model's performance.

11. Interpretability:

Implement interpretability techniques, such as Grad-CAM or LIME, to provide insights into how the model is making predictions. This is crucial for gaining trust in medical applications.

12. Hyperparameter Tuning:

Experiment with hyperparameters like learning rate, batch size, and dropout rate to optimize model performance. Use the validation set to fine-tune these hyperparameters.

13. Early Stopping:

Implement early stopping to prevent overfitting. Monitor the validation loss, and stop training when it starts increasing.

14. Deployment:

Deploy the trained DenseNet-121 model in a real-world application, telemedicine platform, or web interface for users to check their diabetic retinopathy condition.

15. Continuous Monitoring and Updates:
Continuously monitor the model's performance and update it as necessary to maintain efficiency and accuracy in real-world scenarios.

16. Regulatory Compliance:

We have ensured that the model complies with healthcare regulations and patient data privacy laws, especially when handling sensitive medical data.
Building an efficient deep neural network model using DenseNet-121 for the detection of diabetic retinopathy requires careful attention to data, model architecture, training, and evaluation. Regular updates and continuous monitoring are essential to ensure the model's performance and relevance in clinical settings.

1.4 Scope of the project

The scope of this project encompasses a comprehensive approach to the early identification and diagnosis of diabetic retinopathy using a supervised learning model. The project will primarily target the early detection of diabetic retinopathy, reducing the burden on medical professionals and enhancing the speed and accuracy of diagnoses. It will be designed to operate as a supportive tool for healthcare providers and as an aid in the decision-making process. The project's scope acknowledges the potential for continuous improvement, expansion, and adaptation to emerging technologies and healthcare needs.

CHAPTER – 2

2. LITERATURE SURVEY

Diabetic Retinopathy is one of the grave concerns that engrossed the whole world. Receiving the attention from various researchers in order to find the optimal solutions for the early detection of this disease, consequently leading to the prevention of premature fluctuations in vision. Many studies have been conducted and still continues in this field with an aim to ease the lives of both doctors as well as patients. This section provides a review of many research works in the area of Diabetic retinopathy. J.calleja.et al in their work used a two staged method for Diabetic retinopathy detection: LBP (Local Binary Patterns) for feature extraction and Machine Learning specifically SVM and Random Forest for classification purpose. The results obtained by the random forest outperformed the SVM with an accuracy of 97.46%. However, the dataset used in this study was quite small with 71 images. Earlier works were based on manual feature extraction for detection of DR using various computer based systems. U.Acharya.et al used features like blood vessels, microaneurysms, exudates, and haemorrhages from 331 fundus images using SVM with an accuracy of more than 85%. K. Anant.et al in their literature used texture and wavelet features for DR detection by making use of data mining and image processing on a database DIARETDB1 and achieved 97.95% accuracy. M.Gandhi.et al proposed a method for automatic DR detection with SVM classifier by detecting exudates from fundus images. Some works try to integrate manual feature extraction with deep learning feature extraction for DR. one of such work include J.Orlando.et al where CNN with hand crafted feature are used for feature extraction for detecting red lesion in the retina of an eye. S.Preetha et al. In their literature predicted various diabetic related diseases using Data Mining and machine learning methods specifically for heart disease and skin cancer prediction while considering both advantages and disadvantages. While many researches or works are there about using machine learning approaches or data mining approaches, a quite different approach also came into the way of detection of diabetic retinopathy. S.Sadda et al. make use of quantative approach to identify new parameters for detecting proliferative diabetic retinopathy. It is based on the hypothesis that location, number and area of lesions can improve the forecasting process of Retinopathy. The methods used for this study were Subjects and Imaging Data, Ultrawide Field Image Lesion Segmentation, Quantitative Lesion Parameters and Statistical Analysis. Comparison of lesions were made on the basis of Lesion number, Lesion surface area, Lesion distance from the ONH center and Regression analysis. The work presented by J.Amin et al. provides a review of various methodologies for diabetic retinopathy by detecting hemorrhages, microaneurysms, exudates and also blood vessels, and analyzes the various results obtained from theses methodologies experimentally in order to give indepth insight of ongoing research.

The study carried out by Y.Kumaran and C.Patil focuses on the different types of preprocessing

and segmentation ICRIET 2020 IOP Conf. Series: Materials Science and Engineering 1070 (2021) 012049 IOP Publishing doi:10.1088/1757-899X/1070/1/012049 4 techniques mostly and gives an in detail procedure for detection of diabetic retinopathy in human eye consisting of number of systems and classifiers. M.Chetoui et al. Proposes a diagnostic method for DR using machine learning specifically SVM and Texture features. Texture features used were LTP (Local Ternary Pattern) and LESH (Local Energy-based Shape Histogram) that provided better results when compared to Local Binary Pattern (LBP). The accuracy of 90.4% was obtained by LESH with SVM. Deep learning is the most popular approach among researchers for detection, prediction, forecasting and classification task in various fields from few years, in medical field particularly in diabetic retinopathy it is unveiling many possibilities for the prevention of such a dreadful disease. I.Sadek et al. in their work automatically detected the diabetic retinopathy using deep learning approach. They used the four convolutional neural network to classify the diabetic retinopathy into three classes as Normal, Exudates, Drusen. This method outperforms the Bag of words approach and achieved an accuracy of 91%-92%. G.Zago et al. in their study designed a lesion localization model using a deep network specifically convolutional neural network approach with an aim to address the models complexity so, that the performance can be improved. Instead of segmentation localization process of regions were used. Two convolutional networks were used for training purpose on a Standard Diabetic Retinopathy Database and DIARETDB1 where 94% - 95% of sensitivity was obtained. D.Doshi et al. in their work used a GPU based convolutional neural network to classify the retinal images severity level into 5 stages. This approach used 3 models of CNN architecture and an ensemble model of the three to evaluate the results on kappa metrics. The best result were achieved by ensembling method with a score of 0.3996. The work of P.Kaur et al. Presents a Neural network technique for the classification of retinal images using MATLAB.

The results obtained were compared with the machine learning approach like SVM where better results were achieved. M.Voets et al. in their study used a kaggle dataset EyePACS for detection of diabetic retinopathy from retinal fundus images.

CHAPTER-3

3. SOFTWARE REQUIREMENTS SPECIFICATION

In this section we look to describe in detail the requirements we seek from our project in order to be efficient. This description is divided into 4 parts, Functional Requirement Specification, Performance Requirements, Software Requirements, and Hardware Requirements. In Functional Requirement Specification we identify different modules of our system and express the role performed by each module. In Performance Requirements, we broadly outline the requirements in terms of performance from the system. In Software and Hardware Requirements we label the minimum requirements needed for running the project.

3.1 Functional requirement analysis

Functional requirements explain what has to be done by identifying the necessary task, action or activity that must be accomplished. Functional requirements analysis will be used as the top- level functions for functional analysis.

3.2 User requirements analysis

User Requirements Analysis is the process of determining user expectations for a new or modified product. These features must be quantifiable, relevant and detailed. The main user requirements of our project are as follows:

- Internet Facility/ LAN Connection
- CPU i5+
- Provide image
- RAM 8 or 16 GB
- Memory 1GB

3.3 Input design:

Input design is a part of overall system design. The main objective during the input designis as given below:

- To produce a cost-effective method of input.
- To achieve the highest possible level of accuracy.
- To ensure that the input is acceptable and understood by the user.

3.4 Input types:

It is necessary to determine the various types of inputs. Inputs can be categorized asfollows:

- External inputs, which are prime inputs for the system.
- Internal inputs, which are user communications with the system.
- Operational, which are the computer department's communications to the system?
- Interactive, which are inputs entered during a dialogue.

3.5 Output design:

Outputs from computer systems are required primarily to communicate the results of processing to users. They are also used to provide a permanent copy of the results for later consultation. The various types of outputs in general are:

- External Outputs, whose destination is outside the organization.
- Internal Outputs whose destination is within the organization.
- User's main interface with the computer.
- Operational outputs whose use is purely within the computer department.
- Interface outputs, which involve the user in communicating directly.

CHAPTER -4

4. SYSTEM DESIGN DESCRIPTION

4.1 Introduction:

Software design sits at the technical kernel of the software engineering process and is applied regardless of the development paradigm and area of application. Design is the first step in the development phase for any engineered product or system. The designer's goal is toproduce a model or representation of an entity that will later be built. Once system requirements have been specified and analyzed, system design is the first of the three technical activities - design, code and test that is required to build and verify software. The importance can be stated with a single word "Quality". Design is the place where quality is fostered in software development. Design provides us with representations of software that can assess quality.

Design is the only way that we can accurately translate a customer's view into a finished software product or system. Software design serves as a foundation for all the software engineering steps that follow. Without a strong design we risk building an unstable system – one that will be difficult to test, one whose quality cannot be assessed until the last stage. The purpose of the design phase is to plan a solution of the problem specified by the requirement document. This phase is the first step in moving from the problem domain to the solution domain. In other words, starting with what is needed, design takes us toward how to satisfy the needs.

The design of a system is perhaps the most critical factor affecting the quality of the software; it has a major impact on the later phase, particularly testing, maintenance. The output of this phase is the design document. This document is similar to a blueprint for the solution and is used later during implementation, testing and maintenance. The design activityis often divided into two separate phases: System Design and Detailed Design. System Design also called top-level design aims to identify the modules that should be in the system, the specifications of these modules, and how they interact with each other to produce

the desired results.

At the end of the system design all the major data structures, file formats, output formats, and the major modules in the system and their specifications are decided. During Detailed Design, the internal logic of each of the modules specified in system design is decided.

modules, whereas during detailed design the focus is on designing the logic for each of the modules. In other works, in system design the attention is on what components are needed, while in detailed design how the components can be implemented in software is the issue. Design is concerned with identifying software components specifying relationships among components. Specifying software structure and providing a blueprint for the document phase. Modularity is one of the desirable properties of large systems.

It implies that the system is divided into several parts. In such a manner, the interaction between parts is minimal and clearly specified**.** During the system design activities, Developers bridge the gap between the requirements specification, produced duringrequirements elicitation and analysis, and the system that is delivered to the user. Design is the place where the quality is fostered in development. Software design is a process through which requirements are translated into a representation of software.

4.2 UML Design:

UML combines best techniques from data modeling (entity relationship diagrams), business modeling (workflows), object modeling, and component modeling. It can be used with all processes, throughout the software development life cycle, and across different implementation technologies. UML has synthesized the notations of the Bloch method, the Object-modeling technique (OMT) and Object-oriented software engineering (OOSE) by fusing them into a single, common and widely usable modeling language. UML aims to be a standard modeling language which can model concurrent and distributed systems.

4.2.1 Use case Diagram:

Use case diagrams model the functionality of a system using actors and use cases. Usecases are services or functions provided by the system to its users.

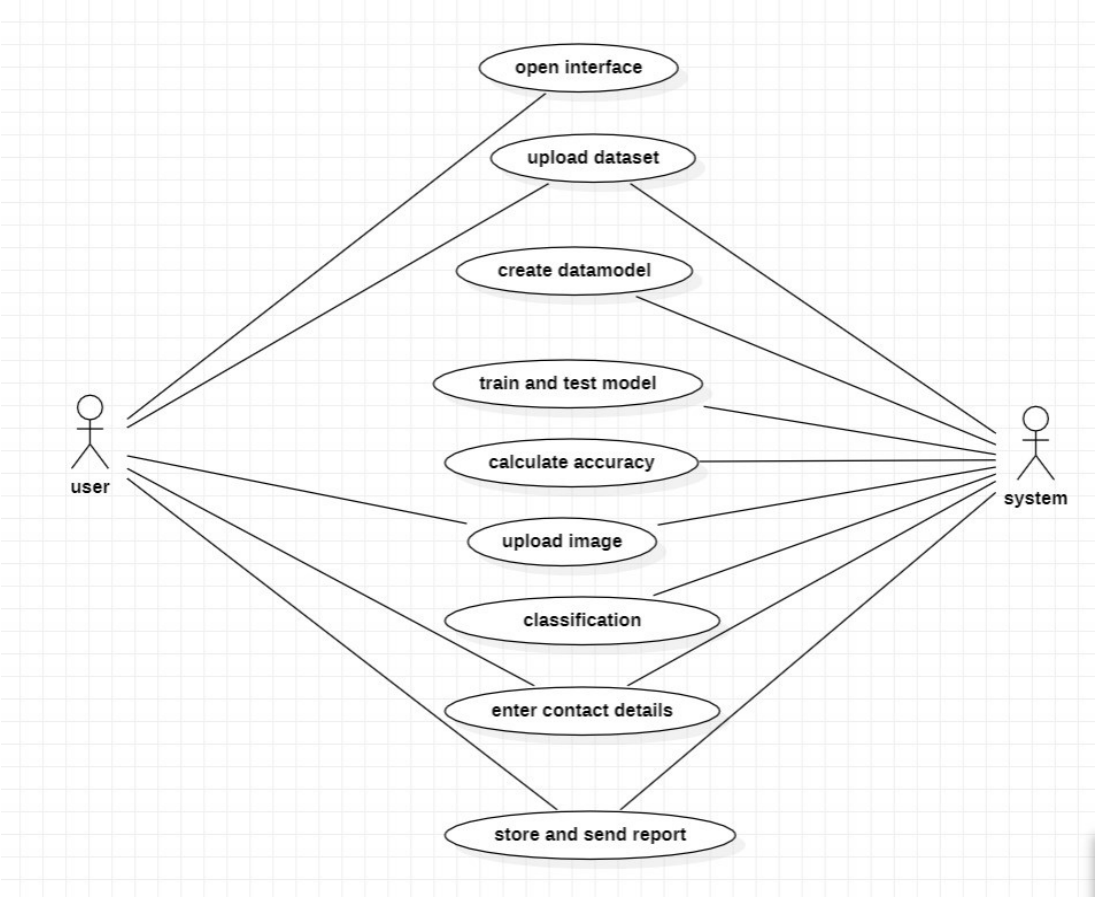


Fig 4.2.1 Use case diagram

4.2.2 Class diagram:

Class diagram is a static diagram. It represents the static view of an application. Classdiagram is not only used for visualizing, describing, and documenting different aspects of asystem but also for constructing executable code of the software application

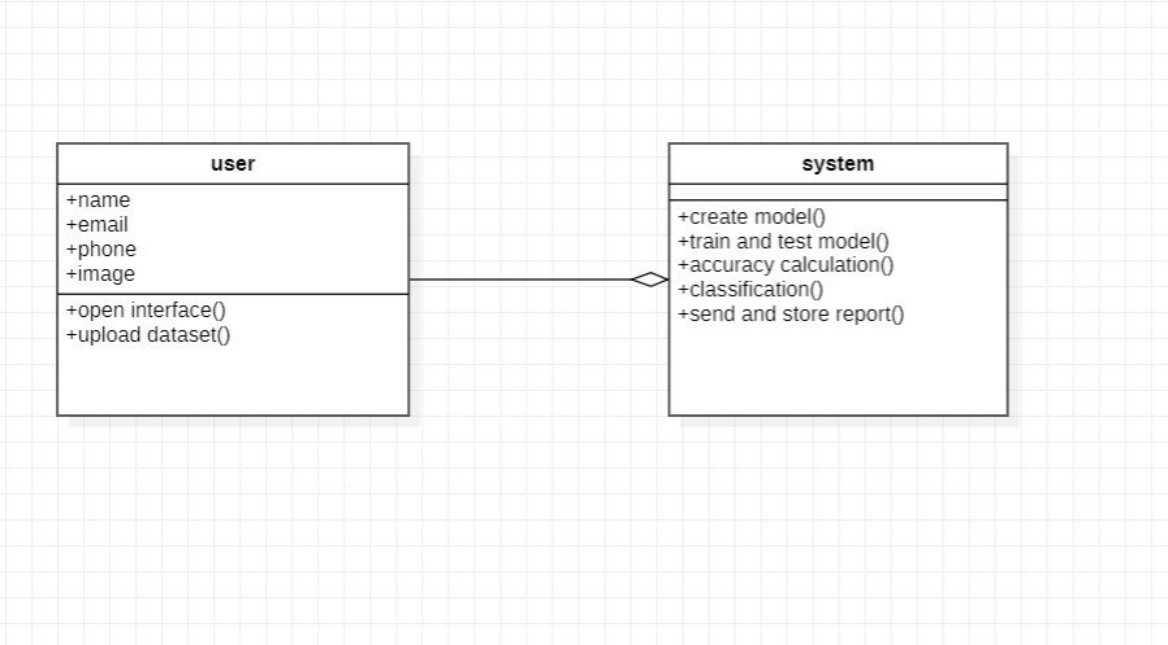


Fig 4.2.2 Class diagram

4.2.3 Sequence diagram:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. A sequence diagram shows, as parallel vertical lines ("lifelines"), different processes or objects that live simultaneously, and, as horizontal arrows,the messages exchanged between them, in the order in which they occur. This allows the specification of simple runtime scenarios in a graphical manner.

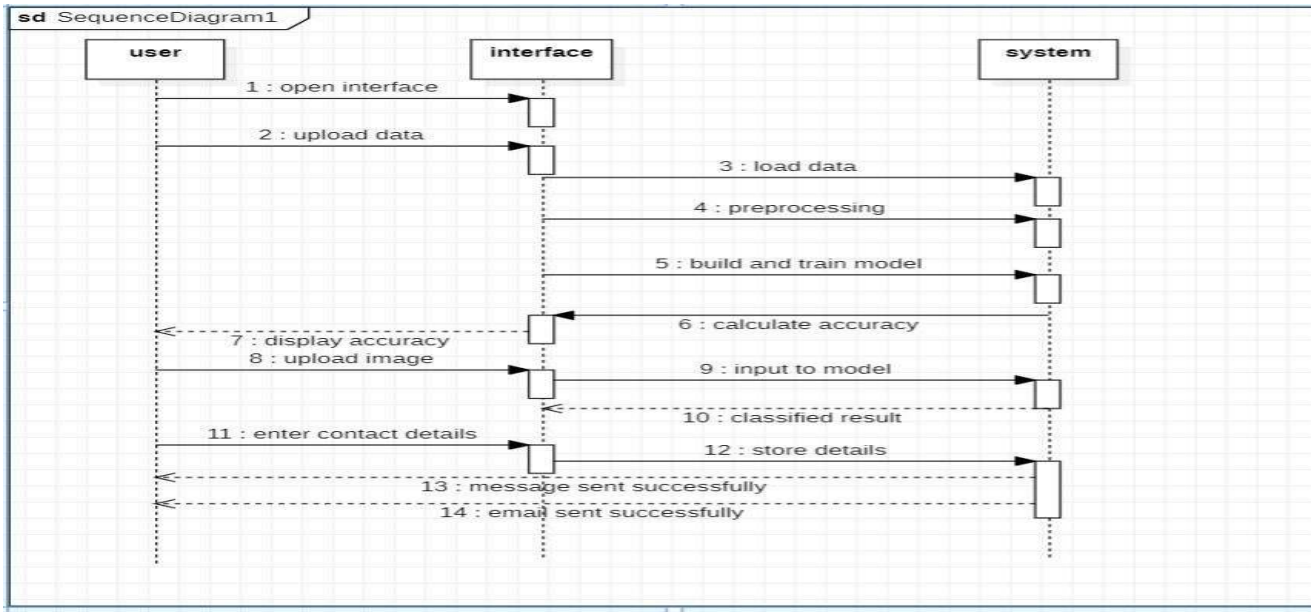


Fig 4.2.3 Sequence diagram

4.2.4 Activity diagram:

Activity diagrams are graphical representations of Workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

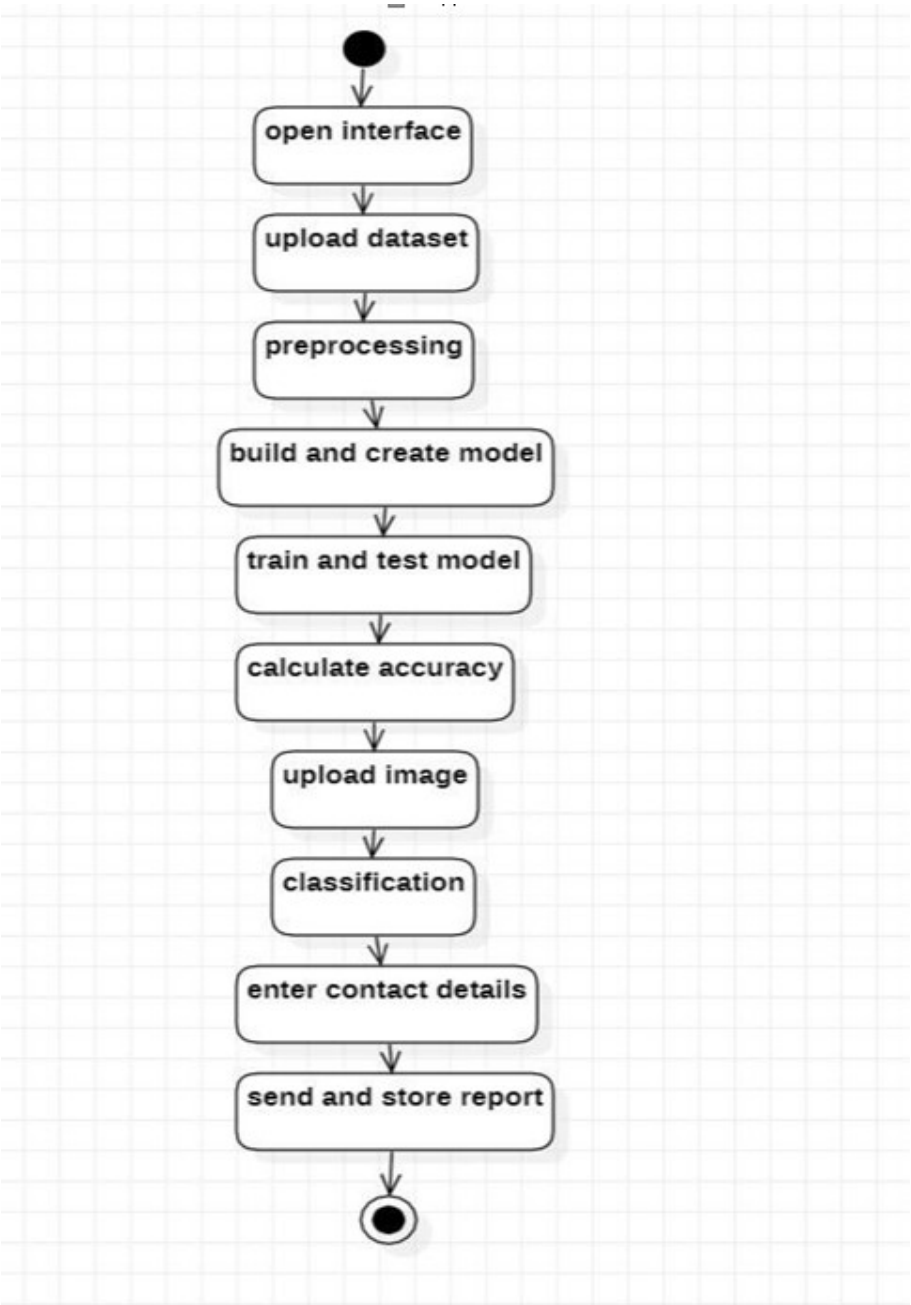
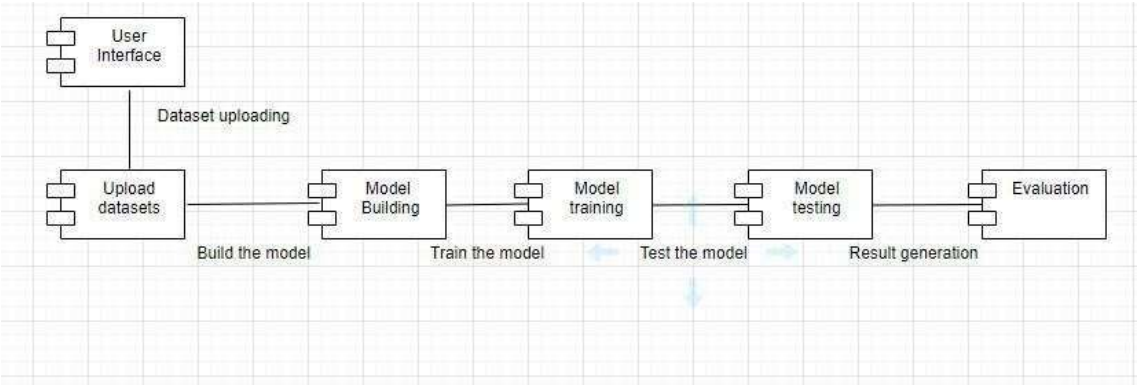


Fig 4.2.4 Activity diagram

4.2.5 Component diagram:

Component diagrams are used to visualize the physical components in a system.



4.2.5 Component diagram

CHAPTER - 5

5. IMPLEMENTATION

5.1 Code Snippets

```
import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

O/P:

```
/kaggle/input/diabetic-retinopathy-detection/train.zip.003
/kaggle/input/diabetic-retinopathy-detection/test.zip.004
/kaggle/input/diabetic-retinopathy-detection/test.zip.005
/kaggle/input/diabetic-retinopathy-detection/train.zip.002
/kaggle/input/diabetic-retinopathy-detection/test.zip.006
/kaggle/input/diabetic-retinopathy-detection/test.zip.003
/kaggle/input/diabetic-retinopathy-detection/train.zip.005
/kaggle/input/diabetic-retinopathy-detection/train.zip.001
/kaggle/input/diabetic-retinopathy-detection/sampleSubmission.csv.zip
/kaggle/input/diabetic-retinopathy-detection/test.zip.007
/kaggle/input/diabetic-retinopathy-detection/trainLabels.csv.zip
/kaggle/input/diabetic-retinopathy-detection/test.zip.001
/kaggle/input/diabetic-retinopathy-detection/sample.zip
/kaggle/input/diabetic-retinopathy-detection/train.zip.004
/kaggle/input/diabetic-retinopathy-detection/test.zip.002
/kaggle/input/prepossessed-arrays-of-binary-data/Binary_images_data_128.npz
/kaggle/input/prepossessed-arrays-of-binary-data/Binary_images_data_90.npz
/kaggle/input/prepossessed-arrays-of-binary-data/1000_Binary_images_data_264.npz
/kaggle/input/prepossessed-arrays-of-binary-data/1000_Binary Dataframe
/kaggle/input/prepossessed-arrays-of-binary-data/Binary_images_data_264.npz
/kaggle/input/prepossessed-arrays-of-binary-data/1000_Binary_images_data_128.npz
/kaggle/input/prepossessed-arrays-of-binary-data/1000_Binary_images_data_90.npz
/kaggle/input/prepossessed-arrays-of-binary-data/Binary Dataframe
```

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
# from keras.preprocessing.image import img_to_array
# from keras.preprocessing.image import array_to_img
# from sklearn.model_selection import train_test_split
# from PIL import Image
# import scipy
```

```
import tensorflow as tf
from tensorflow.keras.applications import *
from tensorflow.keras.optimizers import *
from tensorflow.keras.losses import *
from tensorflow.keras.layers import *
```

```
from tensorflow.keras.models import *
from tensorflow.keras.callbacks import *
from tensorflow.keras.preprocessing.image import *
from tensorflow.keras.utils import *
# import pydot
from sklearn.metrics import *
from sklearn.model_selection import *
import tensorflow.keras.backend as K

# from tqdm import tqdm, tqdm_notebook
# from colorama import Fore
# import json
import matplotlib.pyplot as plt
import seaborn as sns
from glob import glob
from skimage.io import *
%config Completer.use_jedi = False
# import time
# from sklearn.decomposition import PCA
# from sklearn.svm import LinearSVC
# from sklearn.linear_model import LogisticRegression
# from sklearn.metrics import accuracy_score
# import lightgbm as lgb
# import xgboost as xgb
# !pip install livelossplot
# import livelossplot
# from livelossplot import PlotLossesKeras
import warnings
warnings.filterwarnings('ignore')
print("All modules have been imported")
```

O/P:

```
All modules have been imported
```

```
import itertools
def plot_confusion_matrix(cm, classes,
                          normalize=False,
                          title='Confusion matrix',
                          cmap=plt.cm.Blues):
    """
    This function prints and plots the confusion matrix.
    Normalization can be applied by setting normalize=True.
    """
    plt.figure(figsize = (6,6))
    plt.imshow(cm, interpolation='nearest', cmap=cmap)
    plt.title(title)
    plt.colorbar()
    tick_marks = np.arange(len(classes))
    plt.xticks(tick_marks, classes, rotation=90)
    plt.yticks(tick_marks, classes)
```

```
if normalize:
        cm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]

    thresh = cm.max() / 2.
    cm = np.round(cm,2)
    for i, j in itertools.product(range(cm.shape[0]), range(cm.shape[1])):
        plt.text(j, i, cm[i, j],
                 horizontalalignment="center",
                 color="white" if cm[i, j] > thresh else "black")
    plt.tight_layout()
    plt.ylabel('True label')
    plt.xlabel('Predicted label')
    plt.show()
info=pd.read_csv("../input/prepossessed-arrays-of-binary-data/1000_Binary
Dataframe")
info=info.drop('Unnamed: 0',axis=1)
info.head()
```

O/P:

	exists	eye_side	level	path	patient_id	level_cat
0	True	left	0	../input/diabetic-retinopathy-detection/10_lef...	10	[1. 0.]
1	True	right	0	../input/diabetic-retinopathy-detection/10_rig...	10	[1. 0.]
2	True	left	0	../input/diabetic-retinopathy-detection/13_lef...	13	[1. 0.]
3	True	right	0	../input/diabetic-retinopathy-detection/13_rig...	13	[1. 0.]
4	True	left	0	../input/diabetic-retinopathy-detection/17_lef...	17	[1. 0.]

```
sns.set_style('darkgrid')
fig, ax = plt.subplots(figsize=(10,5))
sns.barplot(x=info.level.unique(),y=info.level.value_counts(),palette='Blues_r',ax=ax)
<AxesSubplot:ylabel='level'>

sizes = info['level'].values
sns.distplot(sizes, kde=False)
<AxesSubplot:>

Binary_90 = np.load('../input/prepossessed-arrays-of-binary-
data/1000_Binary_images_data_90.npz')
X_90=Binary_90['a']
Binary_128 = np.load('../input/prepossessed-arrays-of-binary-
data/1000_Binary_images_data_128.npz')
X_128=Binary_128['a']
Binary_264 = np.load('../input/prepossessed-arrays-of-binary-
data/1000_Binary_images_data_264.npz')
X_264=Binary_264['a']
y=info['level'].values
```

```
print(X_90.shape)
print(X_128.shape)
print(X_264.shape)
print(y.shape)
```

O/P:

```
(1000, 24300)
(1000, 49152)
(1000, 209088)
(1000,)
```

```
print("Shape before reshaping X_90" +str(X_90.shape))
X_90=X_90.reshape(1000,90,90,3)
print("Shape after reshaping X_90" +str(X_90.shape))
print("\n\n")
print("Shape before reshaping X_128" +str(X_128.shape))
X_128=X_128.reshape(1000,128,128,3)
print("Shape after reshaping X_128" +str(X_128.shape))
print("\n\n")
print("Shape before reshaping X_264" +str(X_264.shape))
X_264=X_264.reshape(1000,264,264,3)
print("Shape after reshaping X_264" +str(X_264.shape))
Shape before reshaping X_90(1000, 24300)
Shape after reshaping X_90(1000, 90, 90, 3)
Shape before reshaping X_128(1000, 49152)
Shape after reshaping X_128(1000, 128, 128, 3)
Shape before reshaping X_264(1000, 209088)
Shape after reshaping X_264(1000, 264, 264, 3)
plt.title("90*90*3 Image")
plt.imshow(X_90[1])
plt.show()
plt.title("128*128*3 Image")
plt.imshow(X_128[1])
plt.show()
plt.title("264*264*3 Image")
plt.imshow(X_264[1])
plt.show()
y.shape
```

O/P:

```
(1000,)
X=np.array(X_264)
Y=np.array(y)
Y=to_categorical(Y,5)
x_train, x_test1, y_train, y_test1 = train_test_split(X, Y, test_size=0.4,
random_state=42)
x_val, x_test, y_val, y_test = train_test_split(x_test1, y_test1, test_size=0.5,
random_state=42)
```

```
print(len(x_train),len(x_val),len(x_test))
```

O/P:-

600 200 200

DenseNet121 (5 Dense Layer)

```
# Initializing a Sequential model
model = Sequential()
model.add(DenseNet121(input_shape=(264,264,3),include_top=True,weights=None))
model.add(Flatten())
model.add(BatchNormalization())
model.add(Dense(64,kernel_initializer='he_uniform'))
model.add(BatchNormalization())
model.add(Activation('relu'))
model.add(Dropout(0.5))
model.add(Dense(128,kernel_initializer='he_uniform'))
model.add(BatchNormalization())
model.add(Activation('relu'))
model.add(Dropout(0.5))
model.add(Dense(256,kernel_initializer='he_uniform'))
model.add(BatchNormalization())
model.add(Activation('relu'))
model.add(Dropout(0.5))
model.add(Dense(64,kernel_initializer='he_uniform'))
model.add(BatchNormalization())
model.add(Activation('relu'))
model.add(Dropout(0.5))
model.add(Dense(32,kernel_initializer='he_uniform'))
model.add(BatchNormalization())
model.add(Activation('relu'))
model.add(Dropout(0.5))
# Creating an output layer
model.add(Dense(units= 5, activation='softmax'))

c3=tf.keras.callbacks.ReduceLROnPlateau(
   monitor="val_loss",
   factor=0.1,
   patience=2,
   mode="auto",
   min_delta=0.0001,
   cooldown=0,
   min_lr=0.001)
model.compile(optimizer='adam',loss='categorical_crossentropy',
metrics=['accuracy','AUC'])
history=model.fit(x_train,y_train,epochs=20,batch_size=16,validation_split=0.2)
```

O/P:

```
Epoch 1/20
30/30 [==============================] - 23s 253ms/step - loss: 1.9221 - accuracy: 0.2230 - auc: 0.5644 - val_loss: 1.5194 - val_accuracy: 0.2333 - val_auc: 0.8094
Epoch 2/20
30/30 [==============================] - 5s 156ms/step - loss: 1.6859 - accuracy: 0.3039 - auc: 0.6508 - val_loss: 1.4107 - val_accuracy: 0.3417 - val_auc: 0.8354
Epoch 3/20
30/30 [==============================] - 5s 157ms/step - loss: 1.3322 - accuracy: 0.4894 - auc: 0.7820 - val_loss: 1.2827 - val_accuracy: 0.7667 - val_auc: 0.9214
Epoch 4/20
30/30 [==============================] - 5s 157ms/step - loss: 1.2405 - accuracy: 0.5356 - auc: 0.8078 - val_loss: 1.2008 - val_accuracy: 0.7667 - val_auc: 0.9520
Epoch 5/20
30/30 [==============================] - 5s 158ms/step - loss: 1.1540 - accuracy: 0.5565 - auc: 0.8317 - val_loss: 1.0957 - val_accuracy: 0.7667 - val_auc: 0.9416
Epoch 6/20
30/30 [==============================] - 5s 155ms/step - loss: 0.9885 - accuracy: 0.6047 - auc: 0.8769 - val_loss: 1.0064 - val_accuracy: 0.7667 - val_auc: 0.9466
Epoch 7/20
30/30 [==============================] - 5s 157ms/step - loss: 0.9838 - accuracy: 0.6516 - auc: 0.8803 - val_loss: 0.9349 - val_accuracy: 0.7667 - val_auc: 0.9417
Epoch 8/20
30/30 [==============================] - 5s 157ms/step - loss: 0.8871 - accuracy: 0.6294 - auc: 0.8977 - val_loss: 0.8804 - val_accuracy: 0.7667 - val_auc: 0.9436
Epoch 9/20
30/30 [==============================] - 5s 157ms/step - loss: 0.8533 - accuracy: 0.6186 - auc: 0.9046 - val_loss: 0.8177 - val_accuracy: 0.7667 - val_auc: 0.9464
Epoch 10/20
30/30 [==============================] - 5s 157ms/step - loss: 0.8533 - accuracy: 0.6506 - auc: 0.9031 - val_loss: 0.7624 - val_accuracy: 0.7667 - val_auc: 0.9434
Epoch 11/20
30/30 [==============================] - 5s 157ms/step - loss: 0.8440 - accuracy: 0.6435 - auc: 0.9042 - val_loss: 0.7343 - val_accuracy: 0.7667 - val_auc: 0.9469
Epoch 12/20
30/30 [==============================] - 5s 156ms/step - loss: 0.7904 - accuracy: 0.6692 - auc: 0.9149 - val_loss: 0.6940 - val_accuracy: 0.7667 - val_auc: 0.9492
```

```
Epoch 13/20
30/30 [==============================] - 5s 156ms/step - loss: 0.7873 -
accuracy: 0.6843 - auc: 0.9138 - val_loss: 0.6744 - val_accuracy: 0.7667 - val_auc:
0.9526
Epoch 14/20
30/30 [==============================] - 5s 155ms/step - loss: 0.7553 -
accuracy: 0.6608 - auc: 0.9194 - val_loss: 0.6530 - val_accuracy: 0.7667 - val_auc:
0.9452
Epoch 15/20
30/30 [==============================] - 5s 158ms/step - loss: 0.7729 -
accuracy: 0.6477 - auc: 0.9136 - val_loss: 0.6516 - val_accuracy: 0.7667 - val_auc:
0.9441
Epoch 16/20
30/30 [==============================] - 5s 156ms/step - loss: 0.7394 -
accuracy: 0.6867 - auc: 0.9193 - val_loss: 0.6432 - val_accuracy: 0.7667 - val_auc:
0.9444
Epoch 17/20
30/30 [==============================] - 5s 156ms/step - loss: 0.7537 -
accuracy: 0.6623 - auc: 0.9156 - val_loss: 0.6458 - val_accuracy: 0.7667 - val_auc:
0.9393
Epoch 18/20
30/30 [==============================] - 5s 156ms/step - loss: 0.7026 -
accuracy: 0.6507 - auc: 0.9242 - val_loss: 0.6333 - val_accuracy: 0.7667 - val_auc:
0.9453
Epoch 19/20
30/30 [==============================] - 5s 159ms/step - loss: 0.6577 -
accuracy: 0.7202 - auc: 0.9357 - val_loss: 0.6092 - val_accuracy: 0.7667 - val_auc:
0.9451
Epoch 20/20
30/30 [==============================] - 5s 157ms/step - loss: 0.6842 -
accuracy: 0.6937 - auc: 0.9269 - val_loss: 0.5964 - val_accuracy: 0.7667 - val_auc:
0.9419

y_test=np.argmax(y_test, axis=1)
pred=np.argmax(model.predict(x_test),axis=-1)
cm=confusion_matrix(y_test,pred)
cm_plot=plot_confusion_matrix(cm,classes=['0','1'])
print("Performance Report:")
y_pred6=np.argmax(model.predict(x_test),axis=-1)
Y_test=to_categorical(y_test,5)
y_pred_prb6=model.predict(x_test)
target=['0','1']
from sklearn import metrics
print('Accuracy score is :', metrics.accuracy_score(y_test, y_pred6))
print('Precision score is :', metrics.precision_score(y_test, y_pred6,
average='weighted'))
print('Recall score is :',metrics.recall_score(y_test,y_pred6, average='weighted'))
print('F1 Score is :', metrics.f1_score(y_test, y_pred6,average='weighted'))
```

```
print('Cohen Kappa Score:', metrics.cohen_kappa_score(y_test, y_pred6))
print('\t\tClassification Report:\n',
metrics.classification_report(y_test,pred,target_names=target))
```

O/P:

```
Performance Report:
Accuracy score is : 0.785
Precision score is : 0.616225
Recall score is : 0.785
F1 Score is : 0.6904481792717087
Cohen Kappa Score: 0.0
                Classification Report:
              precision    recall  f1-score   support

           0       0.79      1.00      0.88       157
           1       0.00      0.00      0.00        43

    accuracy                           0.79       200
   macro avg       0.39      0.50      0.44       200
weighted avg       0.62      0.79      0.69       200
```

```
import pickle
pickle.dump(rf,open("rf.h5","wb"))
```

Flask Implementation:

```
from _future_ import division, print_function
# coding=utf-8
import sys
import os
import glob
import re
import numpy as np

# Keras
from keras.applications.imagenet_utils import preprocess_input, decode_predictions
from keras.models import load_model
from keras.preprocessing import image

# Flask utils
from flask import Flask, redirect, url_for, request, render_template
from werkzeug.utils import secure_filename
from gevent.pywsgi import WSGIServer

# Define a flask app
app = Flask(_name_)
```

```
# Model saved with Keras model.save()
MODEL_PATH = 'models/my_model.h5'


# Load your trained model
model = load_model(MODEL_PATH)
#model._make_predict_function()          # Necessary
# print('Model loaded. Start serving...')

# You can also use pretrained model from Keras
# Check https://keras.io/applications/
#from keras.applications.resnet50 import ResNet50
#model = ResNet50(weights='imagenet')
#model.save('')
print('Model loaded. Check http://127.0.0.1:5000/')


def model_predict(img_path, model):
    img = image.load_img(img_path, target_size=(224, 224))
    # Preprocessing the image
    x = image.img_to_array(img)
    # x = np.true_divide(x, 255)
    x = np.expand_dims(x, axis=0)
    # Be careful how your trained model deals with the input
    # otherwise, it won't make correct prediction!
    x = preprocess_input(x, mode='caffe')
    preds = model.predict(x)
    return preds


@app.route('/', methods=['GET'])
def index():
    # Main page
    return render_template('index.html')


@app.route('/predict', methods=['GET', 'POST'])
def upload():
    if request.method == 'POST':
        # Get the file from post request
        f = request.files['file']

        # Save the file to ./uploads
        basepath = os.path.dirname(_file_)
        file_path = os.path.join(
            basepath, 'uploads', secure_filename(f.filename))
        f.save(file_path)

        # Make prediction
        preds = model_predict(file_path, model)
```

```
  # Process your result for human
        # pred_class = preds.argmax(axis=-1)            # Simple argmax
        pred_class = decode_predictions(preds, top=1)   # ImageNet Decode

result = str(pred_class[0][0][1])               # Convert to string
        return result
    return None

if _name_ == '_main_':
    app.run(debug=True)
```

CHAPTER - 6

6. TESTING

6.1 Introduction to Testing

Testing is the process of evaluating a system or its component(s) with the intent to find whether it satisfies the specified requirements or not. Testing is executing a system in order to identify any gaps, errors, or missing requirements in contrary to the actual requirements.

According to ANSI/IEEE 1059 standard, Testing can be defined as - A process of analyzing a software item to detect the differences between existing and required conditions (that is defects/errors/bugs) and to evaluate the features of the software item. It depends on the process and the associated stakeholders of the project. In most cases, the following professionals are involved in testing a system within their respective capacities:

- Software Tester
- Software Developer
- Project Lead/Manager
- End User

Levels of testing include different methodologies that can be used while conductingsoftware testing. The main levels of software testing are:

- Functional Testing
- Non-functional Testing

Functional Testing:

This is a type of black-box testing that is based on the specifications of the software that is to be tested. The application is tested by providing input and then the results are examined that need to conform to the functionality it was intended for. Functional testing of a software is conducted on a complete, integrated system to evaluate the system's compliance with its specified requirements.

6.2 Software Testing Life Cycle:

The process of testing a software in a well-planned and systematic way is known as software testing lifecycle (STLC). Different organizations have different phases in STLC however generic Software Test Life Cycle (STLC) for waterfall development model consists of the following phases.

1. Requirements Analysis
2. Test Planning
3. Test Analysis
4. Test Design

- **Requirements Analysis:**

  In this phase testers analyze the customer requirements and work with developers during the design phase to see which requirements are testable and how they are going to test those requirements. It is very important to start testing activities from the requirements phase itself because the cost of fixing defect is very less if it is found in requirements phase rather than in future phases.

- **Test Planning:**

  In this phase all the planning about testing is done like what needs to be tested, how the testing will be done, test strategy to be followed, what will be the test environment, what test methodologies will be followed, hardware and software availability, resources, risks etc. A high-level test plan document is created which includes all the planning inputs mentioned above and circulated to the stakeholders.

- **Test Analysis:**

  After test planning phase is over test analysis phase starts, in this phase we need to dig deeper into project and figure out what testing needs to be carried out in each SDLC phase. If automation needs to be done for software product, how much time will it take to automate and which features need to be automated. Nonfunctional testing areas are also analyzed and defined in this phase.

- **Test Design:**

  In this phase various black-box and white-box test design techniques are used to design the test cases for testing, testers start writing test cases by following those design techniques, if automation testing needs to be done then automation scripts also need to be written in this phase.

6.3 White Box Testing:

We'll be using white box testing for the project in hand since we're using python programming language. White Box Testing is software testing technique in which internal structure, design and coding of software are tested to verify flow of input-output and to improve design, usability and security. In white box testing, code is visible to testers so it is also called Clear box testing, Open box testing, Transparent box testing, Code-based testing and Glass box testing.

What do you verify in White Box Testing?

White box testing involves the testing of the software code for the following:

Internal security holes

Broken or poorly structured paths in the coding processes

The flow of specific inputs through the code

Expected output

The functionality of conditional loops

Testing of each statement, object, and function on an individual basis.

How do you perform White Box Testing?

To give you a simplified explanation of white box testing, we have divided it into two basic steps. This is what testers do when testing an application using the white box testing technique:

Step 1: Understand the source code the first thing a tester will often do is learn and understand the source code of the application. Since white box testing involves the testing of the inner workings of an application, the tester must be very knowledgeable in the programming languages used in the applications they are testing.

Step 2: Create test cases and execute the second basic step to white box testing involves testing the application's source code for proper flow and structure. One way is by writing more code to test the application's source code.

White Box Testing Techniques:

A major White box testing technique is Code Coverage analysis. Code Coverage analysis eliminates gaps in a Test Case suite. It identifies areas of a program that are not

exercised by a set of test cases. Once gaps are identified, you create test cases to verify untested parts of the code, thereby increasing the quality of the software product. There are automated tools available to perform Code coverage analysis. Below are a few coverage analysis techniques 30 Statement Coverage- This technique requires every possible statement in the code to be tested at least once during the testing process of software engineering.

Types of White Box Testing:

White box testing encompasses several testing types used to evaluate the usability of an application, block of code or specific software package. They are listed below: Unit Testing: It is often the first type of testing done on an application. Unit testing is performed on each unit or block of code as it is developed. Unit Testing is essentially done by the programmer. As a software developer, you develop a few lines of code, a single function or an object and test it to make sure it works before continuing Unit Testing helps identify a majority of bugs, early in the software development lifecycle. Bugs identified in this stage arecheaper and easy to fix. Testing for Memory Leaks: Memory leaks are leading causes of slower running applications. A QA specialist who is experienced at detecting memory leaks isessential in cases where you have a slow running software application. Apart from above, a few testing types are part of both black box and white box testing. White Box Penetration Testing: In this testing, the tester/developer has full information of the application's source code, detailed network information, IP addresses involved and all server information the application runs on. The aim is to attack the code from several angles to expose security threats:

ParasoftJt

est

EclEmma

NUnit

PyUnit

HTMLU

nit

CppUnit

Advantages of White Box Testing:

- Code optimization by finding hidden errors.

- White box test cases can be easily automated.
- Testing is more thorough as all code paths are usually covered.
- Testing can start early in SDLC even if GUI is not available.

Disadvantage of white box:

- White box testing can be quite complex and expensive.
- Developers who usually execute white box test cases detest it. The white box testing by developers is not detailed and can lead to production errors.
- White box testing requires professional resources, with a detailed understanding of programming and implementation.

6.4 What is Black Box Testing?

Black box testing is defined as a testing technique in which functionality of the Application under Test (AUT) is tested without looking at the internal code structure, implementation details and knowledge of internal paths of the software. This type of testing is based entirely on software requirements and specifications. In Black Box Testing we just focus on inputs and output of the software system without bothering about internal knowledge of the software program.

How to do Black Box Testing:

Here are the generic steps followed to carry out any type of Black Box Testing.

- Initially, the requirements and specifications of the system are examined.
- Tester determines expected outputs for all those inputs.
- Software tester constructs test cases with the selected inputs.
- The test cases are executed.
- Software tester compares the actual outputs with the expected outputs.
- Defects if any are fixed and re-tested.

Types of Black Box Testing:

There are many types of Black Box Testing but the following are the prominent ones:

Functional testing:

This black box testing type is related to the functional requirements of a system; it isdone by software testers.

Non-functional testing:

This type of black box testing is not related to testing of specific functionality, butnon-functional requirements such as performance, scalability, and usability.

Regression testing:

Regression Testing is done after code fixes, upgrades or any other systemmaintenance to check the new code has not affected the existing code.

Tools used for Black Box Testing:

Tools used for Black box testing largely depend on the type of black box testing youare doing.

- For Functional/Regression Tests you can use -QTP, Selenium
- For Non-Functional Tests, you can use – LoadRunner,J-meter

Black Box Testing Techniques:

Following are the prominent Test Strategy amongst the many used in Black boxTesting:

- Equivalence Class Testing:

  It issued to minimize the number of possible test cases to an optimum level while maintaining reasonable test coverage.

- Boundary Value Testing:

  Boundary value testing is focused on the values at boundaries.

CHAPTER - 7

7. SCREENSHOTS

In this section we shall look at the output screen of various functionalities provided by our project along with the descriptions of each functionality.

```
Out[5]:
        <AxesSubplot:ylabel='level'>
```

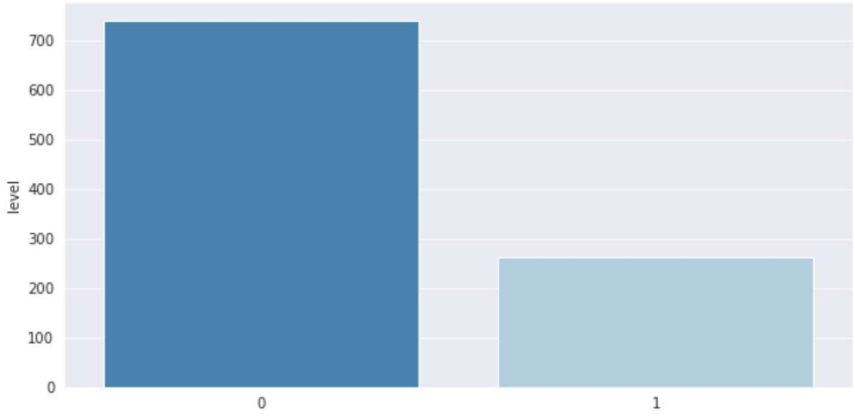


Figure 7.1: Barplot

```
Out[6]:
        <AxesSubplot:>
```

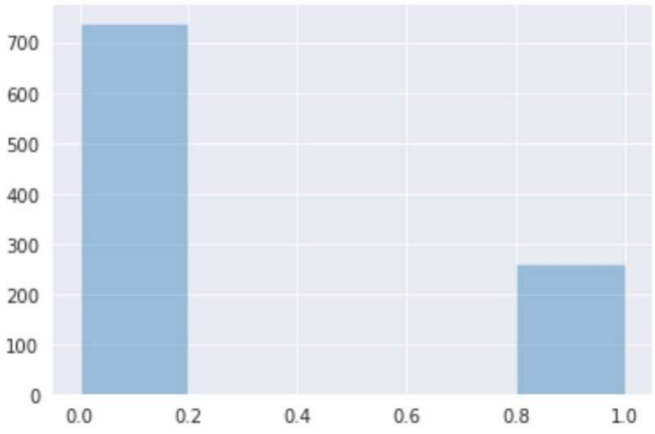


Figure 7.2: Distplot

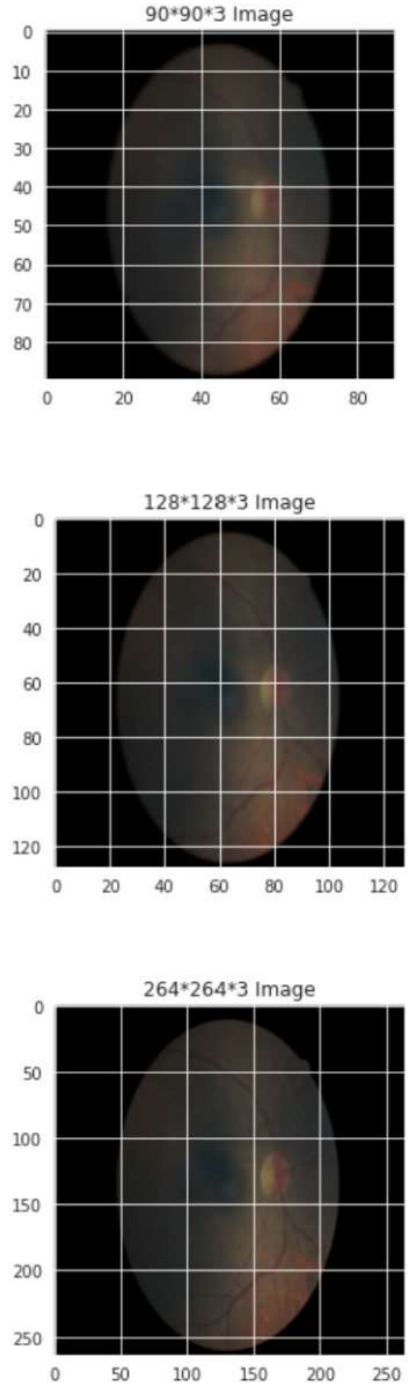


Figure 7.3: Size variation of images

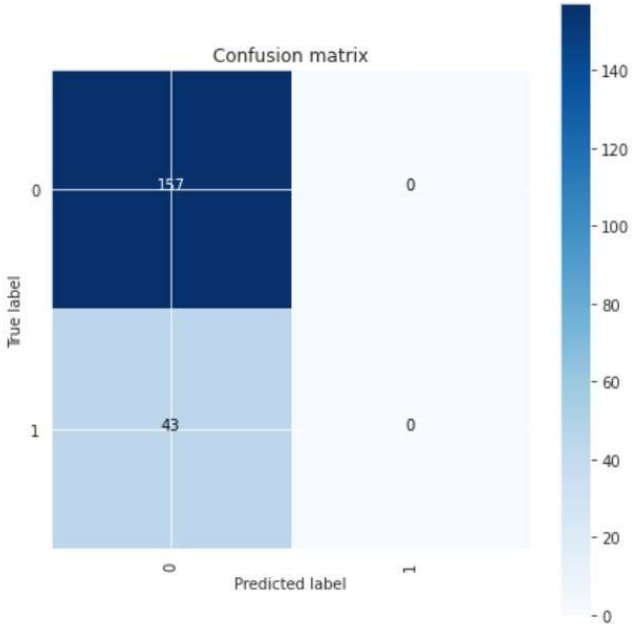


Figure 7.4: Confusion matrix

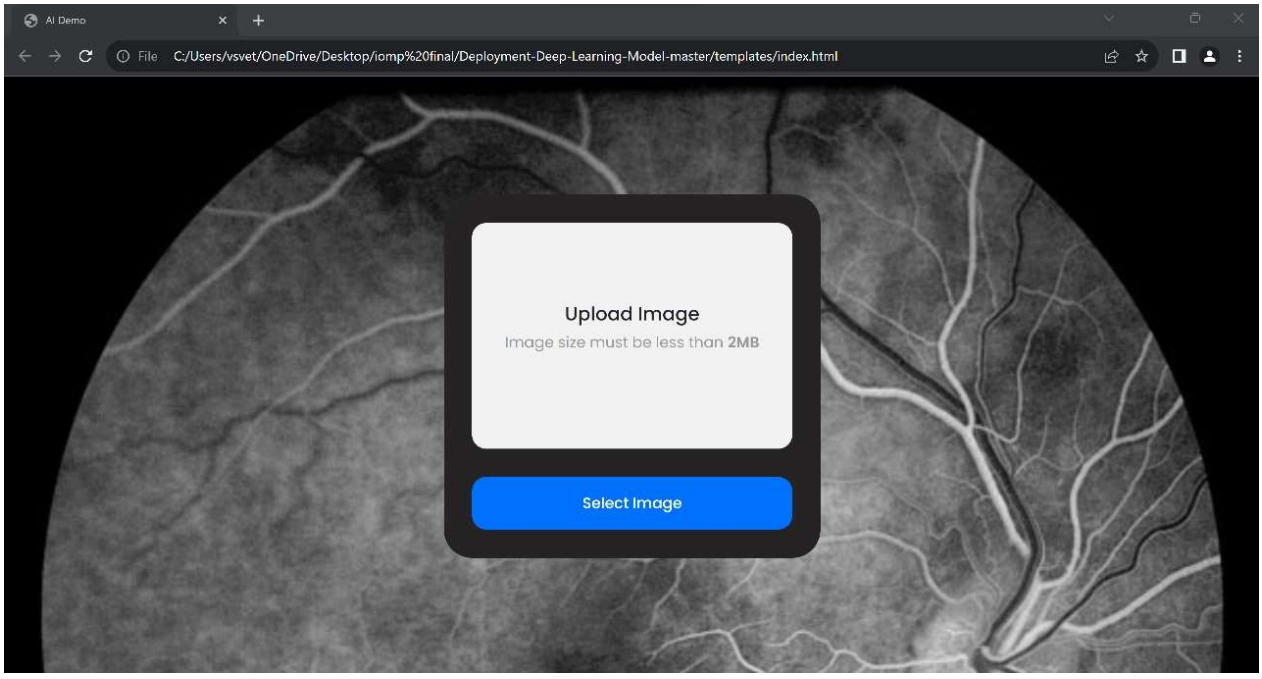


Figure 7.5: Web page UI

CHAPTER-8

8. FUTURE ENHANCEMENT

There is a scope to explore the integration of various data modalities, such as optical coherence tomography (OCT) images, fundus autofluorescence, and patient medical histories, to improve the accuracy of diabetic retinopathy diagnosis. A multimodal approach can provide a more comprehensive view of the condition.

Development of a real-time diagnostic tool that can analyze retinal images on the fly, providing immediate feedback to healthcare professionals during eye examinations is one of many features in future scope of improvements. This can streamline the diagnostic process and lead to quicker patient care decisions. Implement techniques for continuous learning and model adaptation. Over time, the model can adapt to changing patient populations, emerging patterns, and evolving medical knowledge, ensuring that it remains up-to-date and accurate.

CHAPTER - 9

9. CONCLUSION

In this project, we successfully collected a diverse dataset of retinal images, preprocessed the data, and trained a supervised learning model, primarily based on DenseNet 121, to classify diabetic retinopathy with a high degree of accuracy. In conclusion, our project not only contributes to the advancement of medical technology but also holds the potential to improve the lives of individuals living with diabetes. By enabling the early identification of diabetic retinopathy, we aim to prevent the progression of this condition and mitigate the risk of vision loss. However, it is crucial to acknowledge that this project is just a beginning, and there is room for further enhancements and refinements, as well as ongoing collaboration with healthcare professionals to ensure the system's efficacy and ethical compliance.

CHAPTER-10

10.REFERENCES

[1] Wei Zhou, Chengdong Wu, Dali Chen, Zhenzhu Wang, Yugen Yi, Wenyou Du," A NovelApproach for Red Lesions Detection Using Superpixel Multi-Feature Classification in Color Fundus Images", [2017]

[2] Ravitej Singh Rekhi, Ashish Issac, Malay Kishore Dutta Carlos M. Travieso, "Automatedclassification of exudates from digital fundus images", [2017].

[3] Vijay M. Mane, Prof. (Dr.) D.V.Jadhav, Akshay Bansod, "An automatic approach to Hemorrhages detection",[2015]

[4] Amol Prataprao Bhatkar, Dr. G.U.Kharat, "Detection of Diabetic Retinopathy in Retinal Images using MLP classifier"[2015].

[5] Sandra Morales, Kjersti Engan, Valery Naranjo, Adri´an Colomer, "DETECTION OF DIABETIC RETINOPATHY AND AGE-RELATED MACULAR DEGENERATION FROM FUNDUS IMAGES THROUGH LOCAL BINARY PATTERNS AND RANDOM FORESTS" [2015].

[6] Muhammad Nadeem Ashraf a, Zulfiqar Habiba, Muhammad Hussainb, "Texture FeatureAnalysis of Digital Fundus Images for Early Detection of Diabetic Retinopathy" [2016].

[7] Yogesh Kumaran, Chandrashekar M. Patil, "A Brief Review of the Detection of DiabeticRetinopathy in Human Eyes Using Pre-Processing & Segmentation Techniques" [2018]

[8] S. S. Rahim, V. Palade, J. Shuttleworth, and C. Jayne, "Automatic screening and classification of diabetic retinopathy and maculopathy using fuzzy image processing," [2019]

[9] [R.Manjula Sri, V.Rajesh, "Early detection of diabetic retinopathy from retinal fundus images using Eigen value analysis."[2015].

[10] American Diabetes Association. (2021). "Standards of Medical Care in Diabetes—2021." Diabetes Care, 44(Suppl. 1), S1-S232.

[11] Gulshan, V., Peng, L., Coram, M., et al. (2016). "Development and Validation of a Deep Learning Algorithm for Detection of Diabetic Retinopathy in Retinal Fundus Photographs." JAMA, 316(22), 2402-2410.

[12] Ting, D. S. W., Cheung, C. Y.-L., Lim, G., et al. (2017). "Development and Validation of a Deep Learning System for Diabetic Retinopathy and Related Eye Diseases Using Retinal Images From Multiethnic Populations With Diabetes." JAMA, 318(22), 2211-2223.

[13] Abràmoff, M. D., Lou, Y., Erginay, A., et al. (2016). "Improved Automated Detection of Diabetic Retinopathy on a Publicly Available Dataset Through Integration of Deep Learning." Investigative Ophthalmology & Visual Science, 57(13), 5200-5206.

[14] Roychowdhury, S., Koozekanani, D. D., & Parhi, K. K. (2015). "DIFA: A deep learning system for diabetic retinopathy image quality assessment." IEEE Journal of Biomedical and Health Informatics, 20(5), 1477-1485.

[15] Medical Image Analysis for Diabetic Retinopathy: A Comprehensive Review. (2020). Available at: https://pubmed.ncbi.nlm.nih.gov/33005399/